

Opportunity Inbox Copilot — Technical Specification

Tech Stack & Core Philosophy

Backend: Python 3.11 + FastAPI + uvicorn

Frontend: React 18 (Vite), plain CSS

AI: Claude claude-sonnet-4-20250514 via `anthropic` Python SDK

File Handling: `python-multipart` + stdlib `email` module

Total pip installs: `fastapi uvicorn[standard] anthropic python-multipart`

Database: None. All state is in-memory per request.

The single most important design rule: exactly 3 total AI API calls for the entire pipeline. Call 1 classifies all emails in one batch. Call 2 extracts structured fields from all opportunity emails in one batch. Call 3 generates action checklists for all ranked results in one batch. Everything between those three calls is deterministic Python. This gives you fast response time and minimal API cost.

Flow 1: The Ingestion Layer — "The Raw Input Pipeline"

Flow 1 is the entry point. Its sole job is to take messy user input (pasted blobs of text or uploaded files) and normalize everything into a consistent `RawEmail` object before any AI touches it. Nothing downstream knows or cares how the email arrived.

1. The Two Input Sources

Source A: Paste Input

The user pastes multiple emails into a textarea separated by `---EMAIL---`. This delimiter is explicitly shown to the user as an instruction in the UI.

Example paste input:

From: scholarships@hec.gov.pk
Subject: HEC Need-Based Scholarship 2026
Date: April 10, 2026

Applications are now open for the HEC Need-Based Scholarship for BS students.
Minimum CGPA: 2.8. Deadline: May 20, 2026. Apply at hec.gov.pk/scholarships

---EMAIL---

From: noreply@deals.com
Subject: 50% off pizza tonight only!

Get your deal now. Limited time. [Click here.](#)

---EMAIL---

From: icpc@icpcglobal.org
Subject: ICPC Asia West Regional – Registration Open

Teams of 3 are invited to register. Open to all CS undergraduates. No CGPA requirement.

Contest date: June 14, 2026. Register at: icpc.global/asiaw

The backend splits on `---EMAIL---`, giving three raw text segments.

Source B: File Upload

User uploads one or more `.txt` or `.eml` files. Each file = one email.

- `.eml` files: parsed with Python stdlib `email.parser.BytesParser`. Extract `Subject`, `From`, and the `text/plain` part of the body.
- `.txt` files: entire file content treated as the raw email body, subject extracted from the first `Subject:` line if present.

2. The Normalization Step

Both sources produce the exact same output — a `RawEmail` dataclass. The parser function signature looks like:

```
python
```

```
@dataclass
class RawEmail:
    id: str          # "email_001", "email_002", sequentially assigned
    subject: str      # parsed or "No Subject"
    body: str         # full text body, stripped of excess whitespace
    sender: str       # parsed or "Unknown Sender"
    raw_text: str     # original unmodified input, kept for AI extraction
    source: str       # "paste" or "file"
    char_count: int   # used for edge case filtering below
```

Concrete transformation example:

Input text segment (from paste):

```
From: icpc@icpcglobal.org
Subject: ICPC Asia West Regional – Registration Open
Teams of 3 are invited to register...
```

Output `RawEmail`:

```
python

RawEmail(
    id="email_003",
    subject="ICPC Asia West Regional – Registration Open",
    body="Teams of 3 are invited to register. Open to all CS undergraduates...",
    sender="icpc@icpcglobal.org",
    raw_text="From: icpc@icpcglobal.org\nSubject: ICPC Asia West...",
    source="paste",
    char_count=274
)
```

3. Edge Case Handling at Ingestion

Every edge case is caught here before hitting the AI. Fixing it here is free; fixing it after an API call costs a retry.

Edge Case	Detection Condition	Resolution
Email under 30 characters	<code>char_count < 30</code>	Skip it, do not send to classification
No <code>---EMAIL---</code> delimiter in paste	<code>len(split_result) == 1</code>	Treat entire paste as one single email
Encoding error in <code>.eml</code>	<code>UnicodeDecodeError</code> on UTF-8 decode	Re-decode with <code>latin-1</code> fallback
Empty file uploaded	<code>len(content.strip()) == 0</code>	Skip with a logged warning
More than 15 emails submitted	<code>len(emails) > 15</code>	Truncate to the first 15, warn the user in the response
<code>.eml</code> has no <code>text/plain</code> part	<code>get_body(preferencelist= ('plain',))</code> returns None	Fall back to <code>text/html</code> and strip HTML tags with a basic regex
Duplicate emails submitted	Hash of first 200 chars of body matches a seen hash	Keep only the first occurrence, silently deduplicate

4. FastAPI Endpoint

A single endpoint handles both input types:

```
python
@app.post("/api/analyze")
async def analyze(
    pasted_text: Optional[str] = Form(None),
    files: Optional[List[UploadFile]] = File(None),
    profile: str = Form(...)      # JSON string of StudentProfile
)
```

If neither `pasted_text` nor `files` are provided, return HTTP 400 immediately. The AI is never called.

Takeaway: By the end of Flow 1, we have a clean `List[RawEmail]` with consistent fields regardless of input method. Every downstream layer only ever operates on this normalized structure.

Flow 2: The Classification Layer — "Opportunity vs. Noise"

Flow 2 answers a binary question for every email: is this a real student opportunity or not? This is

API Call #1 — all emails classified in a single batch prompt. Never one call per email.

1. The Batch Prompt Structure

All `RawEmail` objects are serialized into a single prompt. Claude is instructed to return a strict JSON array with no preamble. The prompt layout:

```
SYSTEM:
You are an email classifier. Determine if each email contains a real academic
or professional opportunity for a university student.
Real opportunities: scholarships, internships, competitions, fellowships,
admissions, conferences, grants, research positions, jobs.
NOT opportunities: newsletters, promotions, spam, general announcements,
system notifications, payment reminders, social invitations.
Respond ONLY with a valid JSON array. No preamble. No explanation.

USER:
Classify each email below. Return one JSON object per email:
{
  "id": "email_001",
  "is_opportunity": true,
  "category":
"scholarship|internship|competition|fellowship|admission|conference|grant|job|other|
  "confidence": 0.0 to 1.0,
  "reject_reason": "only if is_opportunity is false, else null"
}

EMAIL 1 (id: email_001):
Subject: HEC Need-Based Scholarship 2026
From: scholarships@hec.gov.pk
Body: Applications are now open for the HEC Need-Based Scholarship...

EMAIL 2 (id: email_002):
Subject: 50% off pizza tonight only!
From: noreply@deals.com
Body: Get your deal now. Limited time. Click here.

EMAIL 3 (id: email_003):
Subject: ICPC Asia West Regional — Registration Open
From: icpc@icpcglobal.org
Body: Teams of 3 are invited to register...
```

2. The Classification Response

Claude responds with:

json

```
[
  {
    "id": "email_001",
    "is_opportunity": true,
    "category": "scholarship",
    "confidence": 0.97,
    "reject_reason": null
  },
  {
    "id": "email_002",
    "is_opportunity": false,
    "category": "not_opportunity",
    "confidence": 0.99,
    "reject_reason": "Promotional discount email, not an academic opportunity"
  },
  {
    "id": "email_003",
    "is_opportunity": true,
    "category": "competition",
    "confidence": 0.95,
    "reject_reason": null
  }
]
```

3. Classification Result Dataclass

python

```
@dataclass
class ClassificationResult:
    email_id: str
    is_opportunity: bool
    category: str
    confidence: float
    reject_reason: Optional[str]
```

Only emails where `is_opportunity == True` advance to Flow 3. The filtered-out emails are stored in a separate `filtered_out` list shown at the bottom of the UI.

4. Edge Cases in Classification

Edge Case	Example	Handling
Reminder email	"Reminder: ICPC deadline is tomorrow"	Classify <code>is_opportunity: true</code> — deadline reminders are actionable
Forwarded opportunity	"FWD: HEC Scholarship from your advisor"	Prompt instructs: treat forwarded opportunity emails as real opportunities
JSON parse failure from Claude	Malformed response due to truncation	Try <code>json.loads()</code> in try/except; on failure, conservatively mark all emails <code>is_opportunity: true</code> — never silently discard potential real opportunities
Confidence below 0.6	Vague announcement email	Respect the boolean classification as-is; show "(uncertain)" label next to the item in UI
Category is "other"	Research assistant posting	Treat as opportunity, pass to Flow 3 with category preserved as "other"
Email is about an opportunity but for a different audience	"MBA scholarship for working professionals"	Still classify <code>is_opportunity: true</code> . Profile fit in Flow 4 will penalize it with a low score naturally

Takeaway: After Flow 2, we have two buckets — `opportunities[]` (typically 3-12 emails) and `filtered_out[]`. Only the opportunity bucket moves forward. API call count: **1 of 3 used**.

Flow 3: The Extraction Layer — "Structured Field Parsing"

Flow 3 takes every opportunity email and pulls structured fields out of its natural language. This is **API Call #2** — all opportunity emails extracted in a single batch call.

1. The `OpportunityObject` — The Core Data Structure

Every piece of information downstream scoring needs must be in this one object:

```
python
```

```

@dataclass
class OpportunityObject:
    email_id: str
    title: str # human-readable name, e.g. "HEC Need-Based Scholarship"
    organization: str # "Higher Education Commission Pakistan"
    category: str # from classification layer, carried forward

    # Time
    deadline_raw: str # verbatim: "May 20, 2026" or "rolling" or ""
    deadline_iso: Optional[str] # normalized: "2026-05-20" or None
    deadline_type: str # "explicit" | "inferred" | "rolling" | "missing"

    # Eligibility
    degree_levels: List[str] # ["undergraduate"] or ["any"]
    min_cgpa: Optional[float] # 2.8 or None (None = no requirement)
    skills_required: List[str] # ["Python", "Open Source"] – technical only
    location_type: str # "remote" | "pakistan_only" | "international" | "any"
    financial_need_required: bool
    year_of_study: List[int] # [1,2,3,4] or [] for any semester

    # Documents
    required_documents: List[str] # ["CV", "Transcript", "Cover Letter"]

    # Value signals
    compensation: Optional[str] # "Stipend: $3000" or "Fully funded" or None
    duration: Optional[str] # "12 weeks" or None

    # Contact
    application_link: Optional[str]
    contact_email: Optional[str]

    # Raw text kept for evidence generation
    eligibility_raw: str # verbatim eligibility paragraph from email
    deadline_raw_context: str # the sentence containing the deadline mention

```

2. The Batch Extraction Prompt

SYSTEM:

You are a structured data extractor for academic opportunity emails.
 Extract fields exactly as instructed. If a field cannot be found, use null for
 strings/numbers, [] for lists, false for booleans.
 Never invent information not present in the email.

Rules:

- deadline_iso: convert any date to YYYY-MM-DD. Today is 2026-04-18.

- If deadline is "rolling admissions", set deadline_type="rolling", deadline_iso=null.
- If no deadline found at all, set deadline_type="missing".
- degree_levels: use only these values: ["undergraduate", "graduate", "phd", "any"].
- location_type: use only: "remote", "pakistan_only", "international", "unknown".
- min_cgpa: convert percentage to 4.0 scale using $(\text{pct}/100)*4.0$.
- skills_required: only include specific technical or hard skills (Python, ML, etc).
- Do NOT include soft skills (communication, teamwork).
- If two deadlines exist, use the earlier one as deadline_iso, store both in deadline_raw.
- eligibility_raw: copy the exact sentence(s) from the email that state eligibility.
- deadline_raw_context: copy the exact sentence containing the deadline.

Respond ONLY with a JSON array. No preamble.

USER:

Extract from each email:

OPPORTUNITY 1 (id: email_001, category: scholarship):
[full raw_text of email_001]

OPPORTUNITY 2 (id: email_003, category: competition):
[full raw_text of email_003]

3. Concrete Extraction Example

Input email:

From: scholarships@hec.gov.pk
Subject: HEC Need-Based Scholarship 2026

The Higher Education Commission (HEC) Pakistan is pleased to announce Need-Based Scholarships for deserving undergraduate students enrolled in Pakistani universities.

Eligibility:

- Minimum CGPA: 2.8 on 4.0 scale
- Must demonstrate financial need (family income certificate required)
- Only for BS students currently in Semester 1 through 6

Required Documents: CNIC copy, Official Transcript, Bank statement, Income certificate

Deadline: May 20, 2026

Apply: <https://eportal.hec.gov.pk/scholarship>

Contact: scholarships@hec.gov.pk

Output **OpportunityObject**:

python

```
OpportunityObject(
    email_id="email_001",
    title="HEC Need-Based Scholarship 2026",
    organization="Higher Education Commission (HEC) Pakistan",
    category="scholarship",
    deadline_raw="May 20, 2026",
    deadline_iso="2026-05-20",
    deadline_type="explicit",
    degree_levels=["undergraduate"],
    min_cgpa=2.8,
    skills_required=[],
    location_type="pakistan_only",
    financial_need_required=True,
    year_of_study=[1, 2, 3, 4, 5, 6],
    required_documents=["CNIC copy", "Official Transcript", "Bank statement", "Income certificate"],
    compensation=None,
    duration=None,
    application_link="https://eportal.hec.gov.pk/scholarship",
    contact_email="scholarships@hec.gov.pk",
    eligibility_raw="Minimum CGPA: 2.8. Must demonstrate financial need. Only for BS students currently in Semester 1 through 6",
    deadline_raw_context="Deadline: May 20, 2026"
)
```

4. Edge Cases in Extraction

Edge Case	Example	Rule in Prompt	Output
CGPA as percentage	"Minimum 70% marks required"	Prompt: convert <code>(pct/100)*4.0</code>	<code>min_cgpa: 2.8</code>
Two deadlines	"Early: April 30, Final: May 30"	Prompt: use earlier date as <code>deadline_iso</code>	<code>deadline_iso: "2026-04-30",</code> <code>deadline_raw: "Early: April 30, Final: May 30"</code>
Rolling admissions	"Applications reviewed on a rolling basis"	Prompt: <code>deadline_type="rolling"</code>	<code>deadline_iso: null,</code> <code>deadline_type: "rolling"</code>
Soft skills only	"Strong communication skills required"	Prompt: exclude soft skills	<code>skills_required: []</code>
Vague compensation	"Attractive stipend provided"	Extract verbatim	<code>compensation: "Stipend (amount unspecified)"</code>
No application link but email given	<code>mailto:apply@org.com</code>	Put in <code>contact_email</code> , not <code>application_link</code>	<code>application_link: null,</code> <code>contact_email: "apply@org.com"</code>
Extraction JSON parse failure	Malformed Claude response	<code>try/except</code> ; fall back to <code>OpportunityObject</code> with all fields set to null/empty — scoring handles missing fields gracefully	Opportunity still shows in output with partial info
Location says "Asia"	"Open to students across Asia"	<code>"international"</code>	<code>location_type: "international"</code>

Takeaway: After Flow 3, every opportunity email is a rich structured `OpportunityObject`. API call count: **2 of 3 used**. Everything from here is deterministic Python.

Flow 4: The Scoring Engine — "Deterministic Ranking Math"

Flow 4 is **zero AI**. Pure Python. It takes each `OpportunityObject` + the `StudentProfile` and produces a score from 0-100, a score breakdown, evidence strings, and an urgency badge. This layer is the core intellectual contribution of the system.

1. The Student Profile Schema

The frontend collects this via a structured form — not free text:

```
python

@dataclass
class StudentProfile:
    name: str                # display only
    degree: str              # "CS", "EE", "BBA", "BS Economics"
    degree_level: str        # "undergraduate" | "graduate" | "phd"
    semester: int            # 1-8
    cgpa: float              # 0.0-4.0
    skills: List[str]         # ["Python", "Machine Learning", "React"]
    preferred_types: List[str] # ["internship", "competition"]
    financial_need: bool
    location_preference: str   # "remote" | "pakistan" | "any"
    past_experience: List[str] # ["ICPC 2025 participant", "Web dev intern"]
```

2. The Four Scoring Dimensions

Dimension 1: Profile Fit — max 40 points

This measures how closely the student's attributes match the opportunity's eligibility requirements.

SUB-SCORE A – CGPA Check (0 or 10 points):

```
if opp.min_cgpa is None:                → 10 (no requirement, no penalty)
elif student.cgpa >= opp.min_cgpa:      → 10 (eligible)
elif student.cgpa >= opp.min_cgpa - 0.2: → 5 (borderline, still show with warning)
else:                                   → 0 (below requirement, add hard warning)
```

SUB-SCORE B – Degree/Year Match (0 or 10 points):

```
if opp.degree_levels is empty OR contains "any": → 10
elif student.degree_level in opp.degree_levels: → 10
else:                                           → 0
```

SUB-SCORE C – Skills Match (0-10 points):

```
if opp.skills_required is empty:            → 8 (no requirement, neutral benefit)
else:
    matched = [s for s in student.skills if s in opp.skills_required]
    score = (len(matched) / len(opp.skills_required)) * 10
```

SUB-SCORE D – Preferred Type Match (0 or 10 points):

```
if opp.category in student.preferred_types: → 10
else:                                       → 5 (still relevant, not preferred)
```

Profile Fit Total = A + B + C + D

Dimension 2: Urgency — max 30 points

python

```
TODAY = date(2026, 4, 18) # hardcoded today for scoring, replaced with datetime.to

days_left = (date.fromisoformat(opp.deadline_iso) - TODAY).days if opp.deadline_iso

if days_left is None and opp.deadline_type == "rolling": score = 8
if days_left is None and opp.deadline_type == "missing": score = 5
if days_left < 0:    score = 0    # expired
if days_left <= 7:   score = 30
if days_left <= 14:  score = 25
if days_left <= 30:  score = 20
if days_left <= 60:  score = 12
if days_left > 60:   score = 6
```

Dimension 3: Completeness — max 20 points

Rewards emails that give the student everything needed to actually apply. Pure field presence check.

```
+5 if opp.application_link is not None
+5 if opp.deadline_iso is not None (has an explicit, parseable deadline)
+5 if len(opp.required_documents) > 0
+5 if opp.contact_email is not None OR opp.compensation is not None
```

Dimension 4: Opportunity Value — max 10 points

```
python
VALUE_MAP = {
    "scholarship": 10,
    "fellowship": 10,
    "grant": 7,
    "internship": 8,
    "job": 8,
    "competition": 6,
    "admission": 6,
    "conference": 4,
    "other": 4,
}
```

Final total:

```
total_score = profile_fit + urgency + completeness + value
```

3. Concrete Scoring Example

Student Profile:

```
python
StudentProfile(
    degree="CS", degree_level="undergraduate", semester=6, cgpa=3.3,
    skills=["Python", "Machine Learning", "React"],
    preferred_types=["internship", "competition"],
    financial_need=True, location_preference="any",
    past_experience=["ICPC 2025 participant"]
)
```

Opportunity A: HEC Need-Based Scholarship (*min_cgpa=2.8, degree_levels=["undergraduate"], skills_required=[], financial_need_required=True, deadline_iso="2026-05-20", category="scholarship"*)

Dimension 1 – Profile Fit:

CGPA: 3.3 $\geq$ 2.8 $\rightarrow$ 10/10
Degree: undergraduate in ["undergraduate"] $\rightarrow$ 10/10
Skills: empty requirement $\rightarrow$ 8/10
Type: "scholarship" not in preferred $\rightarrow$ 5/10
Subtotal: 33/40

Dimension 2 – Urgency:

days_left = (2026-05-20) - (2026-04-18) = 32 days $\rightarrow$ 20/30

Dimension 3 – Completeness:

has link +5, has deadline +5, has docs +5, has contact +5 $\rightarrow$ 20/20

Dimension 4 – Value:

"scholarship" $\rightarrow$ 10/10

TOTAL: 33 + 20 + 20 + 10 = 83/100

Opportunity B: ICPC Asia West Regional (*min_cgpa=None, degree_levels=["any"], skills_required=[], deadline_iso="2026-06-14", category="competition"*)

Dimension 1 – Profile Fit:

CGPA: no requirement $\rightarrow$ 10/10
Degree: any $\rightarrow$ 10/10
Skills: empty requirement $\rightarrow$ 8/10
Type: "competition" in preferred $\rightarrow$ 10/10
Subtotal: 38/40

Dimension 2 – Urgency:

days_left = (2026-06-14) - (2026-04-18) = 57 days $\rightarrow$ 12/30

Dimension 3 – Completeness:

has link +5, has deadline +5, no docs 0, no contact/comp 0 $\rightarrow$ 10/20

Dimension 4 – Value:

"competition" $\rightarrow$ 6/10

TOTAL: 38 + 12 + 10 + 6 = 66/100

HEC Scholarship ranks above ICPC ($83 > 66$) because it is more urgent and more complete, despite ICPC being a preferred category.

4. Evidence String Generation — Deterministic Python

Evidence strings are built directly from the scoring logic. No AI.

python

```
def generate_evidence(opp, profile, scores) -> List[str]:
    evidence = []

    # CGPA evidence
    if opp.min_cgpa and scores.cgpa_score == 10:
        evidence.append(f"CGPA eligible: your {profile.cgpa} meets the {opp.min_cgpa}")
    elif opp.min_cgpa and scores.cgpa_score == 0:
        evidence.append(f"WARNING - CGPA below requirement: {profile.cgpa} < {opp.min_cgpa}")
    elif opp.min_cgpa is None:
        evidence.append("No CGPA requirement for this opportunity")

    # Skills evidence
    if opp.skills_required:
        matched = [s for s in profile.skills if s in opp.skills_required]
        evidence.append(f"Skills match: {len(matched)}/{len(opp.skills_required)} relevant")

    # Urgency evidence
    if days_left is not None:
        if days_left <= 7:
            evidence.append(f"HIGH URGENCY: deadline in {days_left} day(s) ({opp.deadline_raw})")
        else:
            evidence.append(f"Deadline: {opp.deadline_raw} ({days_left} days from today)")
    elif opp.deadline_type == "rolling":
        evidence.append("Rolling admissions - apply sooner rather than later")

    # Financial need match
    if opp.financial_need_required and profile.financial_need:
        evidence.append("Financial need requirement matches your profile")

    # Location match
    if opp.location_type == "remote" and profile.location_preference in ["remote", "hybrid"]:
        evidence.append("Remote - no relocation required")

    return evidence
```

Concrete evidence output for HEC Scholarship (student above):

python


```
[
    "CGPA eligible: your 3.3 meets the 2.8 minimum",
    "No skills requirement for this opportunity",
    "Deadline: May 20, 2026 (32 days from today)",
    "Financial need requirement matches your profile"
]
```

5. Urgency Badge Assignment

python

```
def assign_badge(days_left, deadline_type) -> str:
    if days_left is None and deadline_type == "rolling": return "ROLLING"
    if days_left is None: return "NO_DEADLINE"
    if days_left < 0: return "EXPIRED"
    if days_left <= 7: return "RED"
    if days_left <= 30: return "YELLOW"
    return "GREEN"
```

6. The `ScoredOpportunity` Object

python

```
@dataclass
class ScoredOpportunity:
    opportunity: OpportunityObject
    total_score: int
    profile_fit: int
    urgency_score: int
    completeness_score: int
    value_score: int
    evidence: List[str]
    urgency_badge: str # "RED" | "YELLOW" | "GREEN" | "EXPIRED" | "ROLLING" |
    is_ineligible: bool # True if cgpa_score == 0 (hard CGPA fail)
    days_left: Optional[int]
```

Ineligible opportunities (hard CGPA fail) are **not removed** from output — they are sorted to the bottom and shown with a warning. The student deserves to see them and decide.

Ranking is simply:

python

```
ranked = sorted(scored, key=lambda x: (not x.is_ineligible, x.total_score), reverse=
```

Takeaway: After Flow 4, every opportunity has a score 0–100, a ranked position, evidence strings, and an urgency badge — all deterministic Python. API call count still at **2 of 3**. The ranking is complete.

Flow 5: Output Assembly Layer — "Evidence + Checklist Generation"

Flow 5 is **API Call #3 and the last one**. The ranking is already final. Claude here does exactly one thing: write 2–3 practical action bullet points per opportunity. Pure Python cannot generate context-aware human instructions. Claude can.

1. What This Layer Does NOT Do

It does not re-rank. It does not re-score. It does not re-classify. The order determined by Flow 4 is immutable. Claude is a writing tool here, not a reasoning tool.

2. The Batch Checklist Prompt

SYSTEM:

You are a concise student advisor. For each opportunity given, write exactly 2–3 practical next-step action bullets specific to that opportunity.

Rules: max 12 words per bullet. Be specific — mention exact document names, links, or deadlines where available. No generic advice like "check the website."

Respond ONLY with a JSON array. No preamble.

USER:

Generate action checklists for these ranked opportunities:

```
[
  {
    "id": "email_001",
    "title": "HEC Need-Based Scholarship 2026",
    "deadline": "May 20, 2026 (32 days)",
    "required_documents": ["CNIC copy", "Transcript", "Bank statement", "Income certificate"],
    "application_link": "https://eportal.hec.gov.pk/scholarship",
    "category": "scholarship",
    "compensation": null
  },
  {
    "id": "email_003",
    "title": "ICPC Asia West Regional 2026",
    "deadline": "June 14, 2026 (57 days)",
    "required_documents": [],
    "application_link": "https://icpc.global/asiaw",
    "category": "competition",
    "compensation": null
  }
]
```

```
}
]
```

Return: [{"id": "email_001", "checklist": ["step 1", "step 2"]}, ...]

3. Claude Response Example

```
json
[
  {
    "id": "email_001",
    "checklist": [
      "Collect income certificate and bank statement from family now",
      "Request official transcript from FAST registrar office",
      "Apply at eportal.hec.gov.pk/scholarship before May 20"
    ]
  },
  {
    "id": "email_003",
    "checklist": [
      "Form a team of 3 CS students with competitive programming experience",
      "Register your team at icpc.global/asiaw before June 14"
    ]
  }
]
```

4. Fallback if API Call #3 Fails

If the checklist call fails (network timeout, rate limit), generate fallback checklists from structured data with zero API calls:

```
python
def fallback_checklist(opp: OpportunityObject) -> List[str]:
    steps = []
    if opp.application_link:
        steps.append(f"Apply at {opp.application_link}")
    if opp.required_documents:
        docs = ", ".join(opp.required_documents[:3])
        steps.append(f"Prepare required documents: {docs}")
    if opp.deadline_raw:
        steps.append(f"Submit before {opp.deadline_raw}")
    if not steps:
        steps = ["Review the original email for application instructions"]
    return steps
```

The user never sees an error — they get a slightly less polished checklist instead.

5. The Final Response Object

```
python

@dataclass
class RankedOutput:
    rank: int
    opportunity: OpportunityObject
    total_score: int
    score_breakdown: dict          # {"profile_fit": 33, "urgency": 20, ...}
    evidence: List[str]
    checklist: List[str]
    urgency_badge: str
    is_ineligible: bool
    days_left: Optional[int]

@dataclass
class FinalResponse:
    ranked_opportunities: List[RankedOutput]  # sorted highest to lowest score
    filtered_out: List[dict]                  # [{"email_id": ..., "reason": ...}]
    student_profile: StudentProfile
    total_emails_input: int
    opportunities_found: int
    processing_time_ms: int
```

This single JSON object is everything the React frontend needs to render the full result view.

Complete Data Journey — End to End Example

One email, traced through all five flows:

Input email (pasted by user):

```
Subject: Google Summer of Code 2026 – Applications Open
From: gsoc-noreply@google.com

Google Summer of Code is now accepting applications for open source contributors.
Students must be 18+, enrolled in any university. No CGPA requirement.
Required: project proposal, code sample from a recent project.
Stipend: $3,000 – $6,000 USD depending on location and project.
Duration: 12 weeks (fully remote).
Deadline: April 19, 2026.
Apply: summerofcode.withgoogle.com/get-started
```

python

```
RawEmail(  
    id="email_004",  
    subject="Google Summer of Code 2026 – Applications Open",  
    body="Google Summer of Code is now accepting applications...",  
    sender="gsoc-noreply@google.com",  
    source="paste",  
    char_count=398  
)
```

Flow 2 — Classification (part of batch API call):

json

```
{"id": "email_004", "is_opportunity": true, "category": "internship", "confidence":
```



Flow 3 — Extraction (part of batch API call):

python

```
OpportunityObject(  
    email_id="email_004",  
    title="Google Summer of Code 2026",  
    organization="Google",  
    category="internship",  
    deadline_raw="April 19, 2026",  
    deadline_iso="2026-04-19",  
    deadline_type="explicit",  
    degree_levels=["any"],  
    min_cgpa=None,  
    skills_required=[],  
    location_type="remote",  
    financial_need_required=False,  
    required_documents=["project proposal", "code sample"],  
    compensation="Stipend: $3,000-$6,000 USD",  
    duration="12 weeks",  
    application_link="https://summerofcode.withgoogle.com/get-started",  
    contact_email=None,  
    eligibility_raw="Students must be 18+, enrolled in any university. No CGPA requi",  
    deadline_raw_context="Deadline: April 19, 2026."  
)
```



Flow 4 — Scoring (deterministic Python):

```
days_left = (2026-04-19) - (2026-04-18) = 1 day
```

Dimension 1 – Profile Fit:

```
CGPA: no requirement      → 10/10
Degree: "any"             → 10/10
Skills: empty requirement → 8/10
Type: "internship" preferred → 10/10
Subtotal: 38/40
```

Dimension 2 – Urgency:

```
1 day left                → 30/30
```

Dimension 3 – Completeness:

```
link +5, deadline +5, docs +5, compensation counts +5 → 20/20
```

Dimension 4 – Value:

```
"internship"              → 8/10
```

```
TOTAL: 38 + 30 + 20 + 8 = 96/100
```

Evidence:

- "No CGPA requirement for this opportunity"
- "HIGH URGENCY: deadline in 1 day (April 19, 2026)"
- "Remote – no relocation required"
- "Internship matches your preferred type"
- "Compensation: Stipend \$3,000-\$6,000 USD"

Badge: RED

Flow 5 — Checklist (part of batch API call):

```
python
```

```
[
    "Submit your project proposal to summerofcode.withgoogle.com TODAY",
    "Attach a code sample from GitHub or a recent class project",
    "Deadline is tomorrow – do not delay"
]
```

Final output card:

RANK #1 | Score: 96/100 | [RED: 1 DAY LEFT]
Google Summer of Code 2026
Google | Internship | Remote | 12 weeks | Stipend \$3,000-\$6,000

Evidence:

- ✓ No CGPA requirement
- ▲ HIGH URGENCY: deadline in 1 day (April 19, 2026)
- ✓ Remote – no relocation required
- ✓ Internship matches your preferred type

Action Checklist:

- Submit your project proposal to summerofcode.withgoogle.com TODAY
- Attach a code sample from GitHub or a recent class project
- Deadline is tomorrow – do not delay

[Apply →] [See Details ▼]

API Call Budget Summary

Call #	What it does	When it fires
Call 1	Classify all N emails (batch)	After ingestion, once
Call 2	Extract fields from all M opportunity emails (batch)	After filtering, once
Call 3	Generate checklists for all M ranked opportunities (batch)	After scoring, once

Three calls total. No exceptions. No per-email loops.